Study Notes: Days 15–21 — SOLID + Design Patterns

These notes follow Month 1, Week 3 of the roadmap. The C# examples are independent, runnable console examples unless marked as an application fragment. Design examples are illustrative, not claims about your projects. Array complexities use `n` for input length unless otherwise specified.

Day 15 — SOLID: SRP, OCP

Month 1 — Foundations, Week 3 — SOLID + Design Patterns.

Goal: Identify responsibilities by their reasons for change and create useful extension points without adding speculative abstractions.

Single Responsibility Principle (SRP)

SRP says a module should have one primary responsibility, often expressed as one **reason to change**. The useful unit is a coherent business or technical concern—not "a class must have one method." For example, calculating an invoice total, formatting a PDF, and sending email are different responsibilities because pricing rules, document layout, and delivery requirements can change independently and may be owned by different stakeholders.

Separate these concerns behind meaningful boundaries so that a change in formatting does not force changes to pricing logic. Keep related behavior together when it protects the same invariant: splitting every line into a service makes code harder to follow and can spread business rules across the system. A senior-level discussion should identify the actors or policies that drive change, show the coupling in the current design, and explain how tests and deployment boundaries are affected.

SRP applies at multiple scales: methods, types, components, and services. A class with many methods can still be cohesive, while a two-method class can mix unrelated responsibilities. A separate class is not automatically a separate microservice; introducing network boundaries also introduces latency, operational cost, and consistency concerns.

Open/Closed Principle (OCP)

OCP encourages designs that are **open for extension but closed for modification** with respect to a likely kind of change. If a checkout process supports pricing policies that genuinely vary, it can depend on a pricing contract; a new policy can be added without editing the checkout calculation. The client selects or receives the implementation through composition.

The principle is relative to an anticipated axis of change. Adding a policy still requires new code and usually registration or selection logic somewhere. Changing the contract, fixing a defect, or introducing a new dimension of behavior may require modifying existing code. A switch is often the clearest option for a small, stable set of cases. Introduce a polymorphic extension point when it removes repeated edits, supports independently varying behavior, or makes testing and ownership clearer—not merely to avoid any `if`.

The contract must describe behavior, failures, and invariants well enough that implementations are substitutable. Too many extensions can fragment a simple workflow; measure complexity and test each policy

and its integration. OCP works with SRP: isolate the volatile policy, but do not assume an interface alone creates good design.

Code example

```
using System;

var checkout = new Checkout(new TenPercentDiscount());
var receipt = checkout.CreateReceipt(100m);
Console.WriteLine(ReceiptFormatter.Format(receipt)); // Total: 90.0

interface IDiscount
{
    decimal Apply(decimal amount);
}

sealed class TenPercentDiscount : IDiscount
{
    public decimal Apply(decimal amount) => amount * 0.9m;
}

sealed record Receipt(decimal Total);

sealed class Checkout
{
    private readonly IDiscount discount;
    public Checkout(IDiscount discount) => this.discount = discount;
    public Receipt CreateReceipt(decimal amount) => new(discount.Apply(amount));
}

static class ReceiptFormatter
{
    public static string Format(Receipt receipt) => $"Total: {receipt.Total}";
}
```

The formatter and pricing policy change for different reasons. Another discount can implement `IDiscount`, but some composition code must still select it. A production financial system needs explicit validation, currency, rounding, and policy rules.

DSA problem 1: Two Sum

Problem: Given an integer array and an integer target, return indices of two **distinct** elements whose values sum to the target, or `null` if none exists. Any valid pair is acceptable.

Approach: Scan left to right. Before storing the current value, look for its complement among earlier values; this prevents using the same index twice. Store values in a dictionary for expected constant-time lookups. Compute the complement as `long` so subtracting extreme `int` values does not overflow.

Complexity: Expected O(n) time and O(n) extra space; hash-table worst cases can be worse.

```
using System;
using System.Collections.Generic;

var pair = TwoSum(new[] { 2, 7, 11, 15 }, 9);
Console.WriteLine(pair is { } p ? $"{p.First}, {p.Second}" : "No pair"); // 0, 1
Console.WriteLine(TwoSum(new[] { 3, 3 }, 6)); // (0, 1)

static (int First, int Second)? TwoSum(int[] numbers, int target)
{
    var seen = new Dictionary<long, int>();
    for (int i = 0; i < numbers.Length; i++)
    {
        long complement = (long)target - numbers[i];
        if (seen.TryGetValue(complement, out int earlier))
            return (earlier, i);
        seen[numbers[i]] = i;
    }
    return null;
}
```

Edge cases: Empty or one-element input returns `null`; duplicate values can form a pair at different indices; negative values and zero work; no solution returns `null`. With multiple solutions, this returns the first pair discovered by the scan, not a guaranteed lexicographically smallest pair.

DSA problem 2: Move Zeroes

Problem: Move all zeroes to the end of an array in place, preserving the relative order of nonzero values.

Approach: Write each nonzero value to the next available position, then fill the remaining positions with zero. The write index never exceeds the read index, so overwriting does not destroy unread elements.

Complexity: O(n) time and O(1) additional space.

```
using System;

int[] values = { 0, 1, 0, 3, 12 };
MoveZeroes(values);
Console.WriteLine(string.Join(", ", values)); // 1, 3, 12, 0, 0

static void MoveZeroes(int[] values)
{
    int write = 0;
    for (int read = 0; read < values.Length; read++)
        if (values[read] != 0)
            values[write++] = values[read];

    while (write < values.Length)
        values[write++] = 0;
}
```

Edge cases: Empty, all-zero, and no-zero arrays work; nonzero order stays unchanged. The function mutates its input, which should be explicit in an API or interview.

Likely interview questions

1. What is a “reason to change” in SRP, and how do you identify one?
2. Why does one responsibility not mean one method per class?
3. Give an example where applying SRP too aggressively hurts cohesion.
4. What kind of extension should OCP prepare for, and what still has to change when a new implementation is added?
5. When is a simple switch preferable to a hierarchy of implementations?
6. How would you test the checkout separately from its pricing policies?
7. Why must the Two Sum lookup occur before inserting the current value?
8. How does Move Zeroes preserve the relative order of nonzero values?

Common pitfalls / misconceptions

- Counting methods instead of identifying independently changing responsibilities.
- Treating every future requirement as a reason to introduce an interface.
- Assuming OCP means no code or configuration ever changes.
- Returning the same array index twice in Two Sum or changing nonzero order in Move Zeroes.

Cheat-sheet recap

- SRP: group cohesive behavior; separate independently changing concerns.
- OCP: design around real extension axes; avoid speculative frameworks.
- Two Sum: complement lookup before insert; expected O(n) time and O(n) space.
- Move Zeroes: compact nonzero values, then fill zeroes; O(n) time and O(1) space.

Day 16 — SOLID: LSP, ISP, DIP

Month 1 — Foundations, Week 3 — SOLID + Design Patterns.

Goal: Design contracts that can be substituted safely, avoid forcing clients to depend on irrelevant operations, and direct dependencies toward stable policies.

Liskov Substitution Principle (LSP)

LSP says that code using a base type should continue to behave correctly when given a subtype that satisfies the base contract. Matching method signatures is not enough: the subtype must honor the expected behavior, including preconditions, postconditions, invariants, and meaningful failure modes.

A subtype must not demand stricter preconditions than callers were promised, weaken postconditions, or invalidate established invariants. For example, if a withdrawal contract promises to process any positive amount up to the available balance, a subtype that rejects all withdrawals violates that behavioral promise even if it implements the same interface. Design a narrower capability such as `IWithdrawableAccount` when not every account supports withdrawal.

Interviewers often probe the rectangle/square example: if a mutable rectangle promises independent width and height setters, a square that silently changes both dimensions cannot satisfy that contract. The issue is the

mutable API's promises, not mathematics. Favor immutable value types, different abstractions, or composition when substitutability cannot be maintained. Contract tests can exercise shared behavior for all implementations, but they do not prove every possible state is correct.

Interface Segregation Principle (ISP)

ISP says clients should not be forced to depend on members they do not need. A large interface that combines reading, writing, exporting, and administrative operations couples every consumer and implementation to all of those concerns. Split it into cohesive capabilities such as a reader and writer when clients actually vary in what they need.

The benefit is clearer authority, easier tests, and less ripple when one capability changes. Do not mechanically create an interface for every method: excessive fragmentation can make discoverability and orchestration worse. Keep a cohesive interface where consumers naturally need its operations together. A single implementation may implement multiple small interfaces.

Dependency Inversion Principle (DIP)

DIP says high-level policy and low-level details should depend on abstractions rather than high-level policy depending directly on concrete infrastructure. For instance, an order application service can depend on a focused `IOrderReader` contract, while an EF implementation supplies the database details. The application owns the behavior it needs; infrastructure implements the contract, keeping database-specific changes from unnecessarily reshaping the policy.

DIP is not the same as dependency injection. DIP is a dependency-direction principle; dependency injection supplies dependencies from outside an object. A DI container can wire the object graph, but a container alone does not ensure the abstractions are useful. Prefer a contract shaped around the consumer's needs, not a generic interface that merely mirrors every method of a vendor SDK.

Abstractions still carry semantics: transaction boundaries, ordering, consistency, error behavior, and cancellation must be explicit. Over-abstracting can hide provider costs or make valid provider features inaccessible. For small, stable code, direct dependency on a concrete class can be more understandable.

Code example

```
using System;

IOrderReader reader = new InMemoryOrderStore();
var service = new OrderSummary(reader);
Console.WriteLine(service.Describe(1)); // Order 1: 50

interface IOrderReader
{
    Order? Find(int id);
}

interface IOrderWriter
{
    void Save(Order order);
}
```

```
sealed record Order(int Id, decimal Total);

sealed class InMemoryOrderStore : IOrderReader, IOrderWriter
{
    private Order? order = new(1, 50m);
    public Order? Find(int id) => order?.Id == id ? order : null;
    public void Save(Order value) => order = value;
}

sealed class OrderSummary
{
    private readonly IOrderReader reader;
    public OrderSummary(IOrderReader reader) => this.reader = reader;
    public string Describe(int id)
    {
        var order = reader.Find(id);
        return order is null ? "Not found" : $"Order {order.Id}: {order.Total}";
    }
}
```

The summary only needs the reader capability. Any replacement must preserve its contract, including what `null` means. In a real application, an asynchronous interface, cancellation, and a clear consistency contract may be appropriate.

Likely interview questions

1. How can an implementation satisfy an interface syntactically but violate LSP?
2. What do stronger preconditions and weaker postconditions mean in practice?
3. Why is the mutable rectangle/square example a contract problem?
4. How would contract tests help verify multiple implementations?
5. What signals that an interface should be segregated?
6. How can excessive interface splitting make a codebase harder to maintain?
7. How do DIP and dependency injection differ?
8. Where should an abstraction live, and how can it accidentally leak infrastructure details?

Common pitfalls / misconceptions

- Equating inheritance or compilation with behavioral substitutability.
- Implementing unsupported operations by throwing where the contract promised support.
- Applying ISP as “one method per interface.”
- Equating DIP with registering every concrete class in a container.

Cheat-sheet recap

- LSP: preserve behavioral contracts and invariants across substitutions.
- ISP: expose cohesive capabilities that clients actually need.
- DIP: policy depends on useful abstractions, not infrastructure details; DI supplies implementations.

Day 17 — Dependency Injection

Month 1 — Foundations, Week 3 — SOLID + Design Patterns.

Goal: Construct and manage object graphs explicitly, choose appropriate lifetimes, and recognize ownership and concurrency hazards.

Dependency injection: what and why

Dependency injection supplies an object's collaborators from outside instead of having the object construct them internally. Constructor injection makes required dependencies visible and lets the object validate them at creation; method injection is useful for a dependency needed by only one operation, while property injection is generally best reserved for optional or framework-required cases.

DI separates a component's behavior from decisions about concrete implementations and lifecycle. It makes focused tests easier and supports configuration at a composition root. It does not require an interface for every class: inject a concrete, stable collaborator when substitution is not needed. Nor does injecting dependencies eliminate side effects, ensure thread safety, or make a design automatically testable if the contract is too broad.

Container mechanics and object lifetimes

A DI container holds registrations, builds objects and their dependency graphs, and manages owned service lifetimes. In the standard ASP.NET Core container:

- **Transient:** a new instance is created each time the service is resolved.
- **Scoped:** one instance is reused within a scope; a web request normally has a scope.
- **Singleton:** one instance is reused by the root provider across scopes.

These are reuse and ownership rules, not guarantees that an object is thread-safe. Singleton services must safely handle concurrent access and must not capture request-specific state. A scoped service such as an EF Core `DbContext` should not be captured by a singleton: it can escape its intended scope and be used concurrently. Scope validation can detect common cases, but do not rely on it as the only safeguard. A singleton background service can create a scope per unit of work using `IServiceScopeFactory`, resolve scoped services inside it, and dispose the scope after the work completes.

The provider disposes services it creates according to their lifetime; avoid manually disposing container-owned services. If an instance is supplied directly in a registration, clarify ownership because the container does not necessarily own that externally created object. A disposable transient resolved from a long-lived root provider can remain retained until the root is disposed; resolve such work within an appropriate scope.

Composition and tradeoffs

Keep registration near the application entry point. Register the abstraction and implementation, validate configuration at startup where possible, and test that the real graph resolves. Avoid the service-locator pattern—passing `IServiceProvider` deep into business code and resolving arbitrary services at runtime—because it hides dependencies and defers failures. Factories or scoped resolution are appropriate when lifetime or runtime selection genuinely requires them; make that dependency explicit.

Too many constructor dependencies may signal a class with too many responsibilities, although some orchestration components legitimately need several. Decide based on cohesion rather than a fixed count.

Watch for circular dependencies, accidental singleton state, duplicate registrations, and costly construction paths.

Code examples

This standalone example uses constructor injection without a container:

```
using System;

var service = new GreetingService(new ConsoleGreetingWriter());
service.Greet("Sai"); // Hello, Sai

interface IGreetingWriter
{
    void Write(string message);
}

sealed class ConsoleGreetingWriter : IGreetingWriter
{
    public void Write(string message) => Console.WriteLine(message);
}

sealed class GreetingService
{
    private readonly IGreetingWriter writer;
    public GreetingService(IGreetingWriter writer) => this.writer = writer;
    public void Greet(string name) => writer.Write($"Hello, {name}");
}
```

In an ASP.NET Core application, the composition root could register equivalent application types:

```
// Application fragment: builder is a WebApplicationBuilder.
builder.Services.AddScoped<IGreetingWriter, ConsoleGreetingWriter>();
builder.Services.AddScoped<GreetingService>();
```

Here the container creates the dependencies. In real code, choose each lifetime based on its state and resource ownership rather than automatically making everything scoped.

DSA problem 1: Rotate Array

Problem: Rotate an integer array to the right by `k` positions in place. Assume `k` is nonnegative.

Approach: For a nonempty array, reduce `k` modulo `n`, reverse the whole array, reverse the first `k` elements, then reverse the rest. Reversal rearranges the two segments while restoring the order within each segment.

Complexity: O(n) time and O(1) additional space.

```
using System;

int[] values = { 1, 2, 3, 4, 5, 6, 7 };
RotateRight(values, 3);
Console.WriteLine(string.Join(", ", values)); // 5, 6, 7, 1, 2, 3, 4

static void RotateRight(int[] values, int k)
{
    if (k < 0) throw new ArgumentOutOfRangeException(nameof(k));
    if (values.Length == 0) return;

    k %= values.Length;
    Array.Reverse(values);
    Array.Reverse(values, 0, k);
    Array.Reverse(values, k, values.Length - k);
}
```

Edge cases: Empty arrays avoid modulo by zero; `k = 0`, `k` equal to a multiple of `n`, and one-element arrays leave the sequence unchanged. Negative `k` is explicitly rejected; define different behavior if left rotation is required. The input is mutated.

DSA problem 2: Merge Sorted Arrays

Problem: Given two arrays sorted in ascending order, return a new array containing all elements in ascending order, including duplicates.

Approach: Compare the current elements and append the smaller; append the remainder when one array is exhausted. This variant does not assume spare capacity in either input.

Complexity: O(m + n) time and O(m + n) space for the returned array, for input lengths `m` and `n`.

```
using System;

int[] merged = Merge(new[] { 1, 3, 5 }, new[] { 2, 3, 6 });
Console.WriteLine(string.Join(", ", merged)); // 1, 2, 3, 3, 5, 6

static int[] Merge(int[] first, int[] second)
{
    var result = new int[checked(first.Length + second.Length)];
    int i = 0, j = 0, write = 0;
    while (i < first.Length && j < second.Length)
        result[write++] = first[i] <= second[j] ? first[i++] : second[j++];
    while (i < first.Length) result[write++] = first[i++];
    while (j < second.Length) result[write++] = second[j++];
    return result;
}
```

Edge cases: Either or both inputs may be empty; duplicates and negative numbers are retained. The algorithm assumes sorted inputs. An alternative interview variant with spare capacity in the first array can merge

backward in O(1) extra space; its input contract is different.

Likely interview questions

1. How does constructor injection differ from service location?
2. When is an interface unnecessary for an injected collaborator?
3. How do transient, scoped, and singleton lifetimes differ?
4. What happens when a singleton captures a scoped service?
5. How should a background singleton use a scoped dependency?
6. Who disposes container-created services, and why can root-resolved transient disposables be problematic?
7. Why is modulo required in array rotation, and why must the empty case be handled first?
8. How does a new-array merge differ from merging into a first array with spare capacity?

Common pitfalls / misconceptions

- Treating DI and DIP as synonyms or assuming the container makes services thread-safe.
- Capturing a scoped dependency in a singleton or using a scoped `DbContext` concurrently.
- Hiding ordinary dependencies behind `IServiceProvider` in business logic.
- Dividing by array length before handling an empty rotation input; discarding duplicates during merge.

Cheat-sheet recap

- Inject collaborators; compose objects at the application boundary.
- Transient = per resolution; scoped = per scope; singleton = per root provider.
- Match lifetime, disposal, and concurrency requirements.
- Rotate right with three reversals; merge sorted inputs with two pointers.

Day 18 — Factory + Strategy

Month 1 — Foundations, Week 3 — SOLID + Design Patterns.

Goal: Separate object creation from use and select interchangeable algorithms only when the variation is meaningful.

Factory

A factory encapsulates creation of an object. Callers request an appropriate product rather than knowing every concrete type and its construction details. A simple factory can be a method with a switch; the GoF Factory Method specifically uses an overridable creation method, while Abstract Factory creates families of related products. These are related ideas, not interchangeable names for every call to `new`.

Use a factory when creation involves meaningful selection, validation, dependencies, or lifecycle rules, or when callers should not know concrete implementations. For straightforward construction of a stable type, direct `new` or normal DI registration is often clearer. A factory can move a switch to one location, but it does not eliminate the need to update selection or configuration when a new kind is introduced.

Make ownership clear if the factory creates disposable resources. Avoid returning an object whose dependencies outlive or have already lost their required scope. If selection uses external input, reject unsupported values explicitly rather than silently choosing an unintended implementation.

Strategy

Strategy defines a family of interchangeable algorithms behind a common contract and lets a client delegate to the selected one. For example, a shipping calculator can apply a standard or express pricing algorithm without embedding every rule in the workflow. It differs from a factory: the strategy performs behavior; the factory creates or selects an instance.

Strategy is useful when algorithms vary independently, require focused tests, or are selected at runtime. A simple conditional may be better for a small, stable set of trivial cases. The strategy contract must define input validity, output expectations, failures, and state/thread-safety assumptions. Switching algorithms at runtime does not make stateful strategies safe for concurrent singleton use.

Code example

```
using System;

IShippingPrice strategy = ShippingFactory.Create("express");
var checkout = new Checkout(strategy);
Console.WriteLine(checkout.ShippingCost(2)); // 10

interface IShippingPrice
{
    decimal Calculate(int itemCount);
}

sealed class StandardShipping : IShippingPrice
{
    public decimal Calculate(int itemCount) => itemCount * 2m;
}

sealed class ExpressShipping : IShippingPrice
{
    public decimal Calculate(int itemCount) => itemCount * 5m;
}

static class ShippingFactory
{
    public static IShippingPrice Create(string kind) => kind switch
    {
        "standard" => new StandardShipping(),
        "express" => new ExpressShipping(),
        _ => throw new ArgumentException("Unsupported shipping kind.",
nameof(kind))
    };
}

sealed class Checkout
{
    private readonly IShippingPrice shipping;
    public Checkout(IShippingPrice shipping) => this.shipping = shipping;
    public decimal ShippingCost(int itemCount)
    {
```

```
        if (itemCount < 0) throw new
ArgumentOutOfRangeException(nameof(itemCount));
        return shipping.Calculate(itemCount);
    }
}
```

The factory owns selection and the strategy owns calculation. A production implementation should specify pricing, currency, validation, configuration, and resource lifetimes.

Likely interview questions

1. What problem does a factory solve, and when is direct construction simpler?
2. How is a simple factory different from Factory Method?
3. How does Strategy differ from Factory?
4. Where should unsupported selection values be handled?
5. What behavioral guarantees should a strategy interface document?
6. How does dependency injection interact with factories and strategies?
7. What risks arise if a stateful strategy is registered as a singleton?
8. When would introducing a strategy hierarchy be overengineering?

Common pitfalls / misconceptions

- Calling any helper that returns `new` an implementation of GoF Factory Method.
- Expecting a factory to remove every selection or registration change.
- Silently defaulting an invalid kind to a business behavior.
- Using strategies with incompatible rules under a misleading common contract.

Cheat-sheet recap

- Factory decides **how or which object to create**.
- Strategy encapsulates **which interchangeable behavior to execute**.
- Use both when selection and algorithm variation justify the indirection.

Day 19 — Repository + Unit of Work

Month 1 — Foundations, Week 3 — SOLID + Design Patterns.

Goal: Understand data access boundaries and coordinated persistence without pretending they automatically solve transactions or distributed consistency.

Repository

A repository presents access to a collection of domain objects through operations meaningful to its consumers, such as `FindOpenOrder` or `Add`. It can isolate persistence details and centralize queries where that boundary provides value. Its contract should make absence, query limits, tracking, ordering, cancellation, and failure behavior clear.

A generic CRUD wrapper over every ORM method can merely duplicate the ORM and obscure useful features. EF Core's `DbSet<T>` already offers repository-like access; an additional repository is justified when it expresses

a meaningful domain boundary or supports the application's architectural needs. Returning raw `IQueryable<T>` exposes provider translation and context lifetime to callers; that flexibility can be intentional, but it weakens encapsulation and lets callers create expensive or untranslatable queries.

In-memory fakes help test application logic but do not prove that SQL translation, constraints, transaction isolation, or concurrency behavior work. Use provider-backed integration tests for those concerns.

Unit of Work

A Unit of Work tracks changes belonging to one business operation and coordinates persisting them. EF Core's `DbContext` provides change tracking and `SaveChanges`/`SaveChangesAsync`, so it already behaves like a unit of work in many applications. Repositories participating in one unit of work must share the intended context and scope; independently saving inside each repository can prevent a caller from coordinating changes.

With a relational provider, a single `SaveChanges` call is normally transactional when the provider supports transactions. Multiple calls or additional operations may need an explicit transaction, with correct isolation and provider-specific retry behavior. A transaction does not automatically make business decisions race-free: unique constraints, optimistic concurrency tokens, and suitable isolation may still be necessary.

A database transaction cannot automatically atomically include an arbitrary HTTP call or message broker. For reliable database state plus event publication, consider a transactional outbox with a separately delivered message and idempotent consumers. The right guarantee depends on the system; avoid promising exactly-once execution without an end-to-end design.

Code example

This runnable in-memory example illustrates roles, not a durable or transactional database implementation:

```
using System;
using System.Collections.Generic;

var store = new InMemoryOrderStore();
var service = new OrderService(store, store);
service.Place(new Order(1, 25m));
Console.WriteLine(store.Find(1)?.Total); // 25

record Order(int Id, decimal Total);

interface IOrderRepository
{
    void Add(Order order);
    Order? Find(int id);
}

interface IUnitOfWork
{
    void Commit();
}

sealed class InMemoryOrderStore : IOrderRepository, IUnitOfWork
```

```
{
    private readonly Dictionary<int, Order> committed = new();
    private readonly List<Order> pending = new();
    public void Add(Order order) => pending.Add(order);
    public Order? Find(int id) => committed.TryGetValue(id, out var order) ? order
: null;
    public void Commit()
    {
        foreach (var order in pending) committed.Add(order.Id, order);
        pending.Clear();
    }
}

sealed class OrderService
{
    private readonly IOrderRepository orders;
    private readonly IUnitOfWork unitOfWork;
    public OrderService(IOrderRepository orders, IUnitOfWork unitOfWork)
    {
        this.orders = orders;
        this.unitOfWork = unitOfWork;
    }
    public void Place(Order order)
    {
        orders.Add(order);
        unitOfWork.Commit();
    }
}
```

The same store supplies both interfaces so the service's repository and unit of work share state. This simplified implementation can partially modify `committed` if a duplicate key throws during `Commit`; it deliberately does **not** demonstrate atomic rollback, thread safety, persistence, or a production transaction.

DSA problem 1: Missing Number

Problem: An array of length `n` contains distinct numbers from `0` through `n`, with exactly one missing. Return the missing number.

Approach: XOR all numbers from `0` through `n` with all array elements. Equal values cancel because `x ^ x = 0`; the missing value remains. This avoids overflow from calculating a sum.

Complexity: O(n) time and O(1) additional space.

```
using System;

Console.WriteLine(MissingNumber(new[] { 3, 0, 1 })); // 2
Console.WriteLine(MissingNumber(Array.Empty<int>())); // 0

static int MissingNumber(int[] numbers)
{
    int missing = numbers.Length;
```

```
    for (int i = 0; i < numbers.Length; i++)
        missing ^= i ^ numbers[i];
    return missing;
}
```

Edge cases: Empty input represents `n = 0` and returns `0`; the missing value may be `0` or `n`. The method assumes distinct, in-range values and exactly one missing; it does not validate malformed input.

DSA problem 2: Sort 0/1/2

Problem: Sort an array containing only `0`, `1`, and `2` in ascending order, in place.

Approach: Use the Dutch national flag partition. Indices before `low` contain zeroes, indices from `low` to `mid - 1` contain ones, and indices after `high` contain twos. Examine `mid`; swap zero to the front, advance past one, or swap two to the back and examine the new `mid` value again.

Complexity: O(n) time and O(1) additional space.

```
using System;

int[] values = { 2, 0, 2, 1, 1, 0 };
SortColors(values);
Console.WriteLine(string.Join(", ", values)); // 0, 0, 1, 1, 2, 2

static void SortColors(int[] values)
{
    int low = 0, mid = 0, high = values.Length - 1;
    while (mid <= high)
    {
        switch (values[mid])
        {
            case 0:
                (values[low], values[mid]) = (values[mid], values[low]);
                low++;
                mid++;
                break;
            case 1:
                mid++;
                break;
            case 2:
                (values[mid], values[high]) = (values[high], values[mid]);
                high--;
                break;
            default:
                throw new ArgumentException("Values must be 0, 1, or 2.",
nameof(values));
        }
    }
}
```

Edge cases: Empty and single-valued arrays work. Do not increment `mid` after swapping with `high`: the incoming value is unexamined. Invalid input throws, but earlier swaps may already have mutated the array; validate separately first if failure must leave it untouched.

Likely interview questions

1. What does a repository add beyond an ORM, and when is it redundant?
2. What are the tradeoffs of exposing `IQueryable<T>` from a repository?
3. How does EF Core's `DbContext` relate to Unit of Work?
4. Why should repositories in the same unit of work share the relevant context?
5. Is one `SaveChanges` call transactional, and does that solve every concurrency problem?
6. How would you reliably update a database and publish a message?
7. Why does XOR find the missing number, and what input assumptions does it require?
8. Why does the 0/1/2 algorithm re-examine `mid` after swapping with `high`?

Common pitfalls / misconceptions

- Adding a generic repository that only hides useful ORM features without a clear benefit.
- Assuming a mocked repository verifies database translation or transaction semantics.
- Claiming `SaveChanges` atomically includes a remote API or broker.
- Incrementing the middle pointer after moving an unexamined value from the high end.

Cheat-sheet recap

- Repository: meaningful access boundary; Unit of Work: coordinate changes for one operation.
- EF Core already supplies much of both; add abstractions for a reason.
- Database transactions have a scope; external effects need an explicit reliability design.
- Missing Number: XOR cancellation. Sort 0/1/2: three-way partition.

Day 20 — Decorator + Adapter + Facade

Month 1 — Foundations, Week 3 — SOLID + Design Patterns.

Goal: Distinguish wrapping to add behavior, translating an incompatible interface, and simplifying a subsystem.

Decorator

A decorator implements the same contract as the wrapped component, forwards calls to it, and adds behavior before or after delegation. Logging, metrics, and caching are common examples. Multiple decorators can be composed, and their **order matters**: logging outside a cache sees cache hits; logging inside it may see only misses.

Use a decorator when behavior should be applied transparently and independently of the concrete implementation. Avoid it when one direct implementation is simpler or when added behavior changes the contract unexpectedly. Caching introduces freshness, invalidation, failure, and concurrency questions; retries introduce idempotency and load concerns. A wrapper is not automatically safe just because it has the same method signature. Clarify who owns and disposes wrapped resources.

Adapter

An adapter translates between the interface a caller expects and an existing interface that does not match it. For example, an application-facing price reader can convert the units and method names of a legacy price service. The adapter may translate types, errors, and data formats; it should document semantic mismatches such as rounding or unavailable capabilities.

Use an adapter at a boundary to integrate a legacy API, third-party SDK, or protocol without spreading its quirks throughout the application. Do not silently manufacture missing data or swallow errors to make the interfaces appear compatible. If the interfaces already match, an adapter adds unnecessary indirection.

Facade

A facade provides a simpler, task-oriented entry point to a subsystem with several collaborators. It may coordinate validation, inventory checks, and payment authorization behind a `PlaceOrder` operation. It does not need to implement the same contract as any subsystem component and does not inherently change those components' interfaces.

Use a facade when callers repeatedly need to coordinate a complex subsystem and can benefit from a stable, coherent operation. It can coexist with direct subsystem access when needed. A facade can become a god object if it accumulates unrelated workflows; keep responsibilities and transaction boundaries explicit. A multi-step facade is not automatically atomic, especially when it calls external services.

Code example

```
using System;

IPriceReader reader = new LoggingPriceReader(
    new LegacyPriceAdapter(new LegacyCatalog()));
var facade = new QuoteFacade(reader);
Console.WriteLine(facade.Quote("A")); // Reading A; Quote: 12.50

interface IPriceReader
{
    decimal ReadPrice(string productId);
}

sealed class LegacyCatalog
{
    public int GetPriceInCents(string productId) =>
        productId == "A" ? 1250 : throw new ArgumentException("Unknown product.");
}

sealed class LegacyPriceAdapter : IPriceReader
{
    private readonly LegacyCatalog catalog;
    public LegacyPriceAdapter(LegacyCatalog catalog) => this.catalog = catalog;
    public decimal ReadPrice(string productId) =>
catalog.GetPriceInCents(productId) / 100m;
}

sealed class LoggingPriceReader : IPriceReader
{
```

```
    private readonly IPriceReader inner;
    public LoggingPriceReader(IPriceReader inner) => this.inner = inner;
    public decimal ReadPrice(string productId)
    {
        Console.WriteLine($"Reading {productId}");
        return inner.ReadPrice(productId);
    }
}

sealed class QuoteFacade
{
    private readonly IPriceReader prices;
    public QuoteFacade(IPriceReader prices) => this.prices = prices;
    public string Quote(string productId) => $"Quote:
{prices.ReadPrice(productId):F2}";
}
```

The adapter translates cents to decimal units, the decorator adds logging while preserving `IPriceReader`, and the facade exposes a quote operation. In real financial code, specify currency and rounding; avoid logging sensitive identifiers without an appropriate policy.

Likely interview questions

1. How does a decorator differ from an adapter?
2. Why does decorator ordering matter?
3. What behavior can make a caching or retry decorator violate its contract?
4. How should an adapter handle a semantic mismatch or an unsupported operation?
5. What does a facade simplify, and does it make several external calls atomic?
6. When can a facade become a god object?
7. How do you manage disposal when wrappers hold disposable dependencies?
8. When is direct use of a subsystem clearer than introducing one of these patterns?

Common pitfalls / misconceptions

- Naming every wrapper a decorator despite changing the contract or purpose.
- Ignoring decorator ordering, cache invalidation, or retry idempotency.
- Hiding failed translations or manufacturing success-shaped values in adapters.
- Assuming a facade provides a transaction or must replace all access to its subsystem.

Cheat-sheet recap

- Decorator: same contract, added behavior.
- Adapter: translate an incompatible contract or representation.
- Facade: simpler entry point to a subsystem.
- Evaluate semantics, failure handling, ordering, and ownership—not just class diagrams.

Day 21 — Observer + Command + review — for each pattern: what problem, when to use, when NOT to, real example

Month 1 — Foundations, Week 3 — SOLID + Design Patterns.

Goal: Explain Observer and Command, then compare the week's patterns by the problem each solves, its tradeoffs, and a concrete example.

Observer

Observer lets a publisher notify multiple subscribers when something changes without hard-coding every subscriber into the publisher. In C#, events and delegates commonly implement in-process notifications. A publisher raises an event; subscribed handlers receive it. This is useful for UI notifications or decoupled in-process reactions, especially when subscribers have independent responsibilities.

The indirection has costs. Execution order and error handling matter: ordinary synchronous event invocation calls handlers in invocation-list order, and an unhandled exception stops subsequent handlers. The publisher holds references to subscriber targets, so a long-lived publisher can keep subscribers alive unless subscriptions are managed. Events are not durable messages: a process crash can lose notifications, and an event does not automatically cross service boundaries.

Avoid Observer when a direct call is clearer, the operation requires an immediate coordinated result, or durable delivery is required but no delivery mechanism exists. An async event handler using `async void` is not naturally awaitable by the publisher; if async completion and errors matter, design an explicit async contract and define how handlers are awaited. Reliable cross-process publication needs infrastructure and a delivery strategy, not just a C# event.

Command

Command represents an intended action as a value or object, separating the request from the code that invokes or processes it. A command can carry parameters, be passed to a handler, and enable validation, scheduling, auditing, or queuing. In a simple application a direct method call may be clearer; use command objects when the request needs its own identity, handling pipeline, or execution policy.

A command expresses an instruction, often expected to have one responsible handler; an event records that something happened and may have many observers. These are conventions, not guarantees provided by a class name. Commands are not automatically durable, transactional, undoable, or idempotent. If retrying a payment-like operation, define an idempotency key and enforce the intended behavior at an appropriate durable boundary. Undo requires explicitly designed compensating actions or stored prior state; not every real-world side effect can be reversed.

Code example

```
using System;

var account = new Account();
EventHandler<DepositEventArgs> handler =
    (_, e) => Console.WriteLine($"Deposited {e.Amount}");
account.Deposited += handler;

var handlerForDeposit = new DepositHandler(account);
handlerForDeposit.Handle(new DepositCommand(25m));
Console.WriteLine(account.Balance); // 25
account.Deposited -= handler;
```

```
sealed record DepositCommand(decimal Amount);

sealed class DepositEventArgs : EventArgs
{
    public DepositEventArgs(decimal amount) => Amount = amount;
    public decimal Amount { get; }
}

sealed class Account
{
    public decimal Balance { get; private set; }
    public event EventHandler<DepositEventArgs>? Deposited;

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        Balance += amount;
        Deposited?.Invoke(this, new DepositEventArgs(amount));
    }
}

sealed class DepositHandler
{
    private readonly Account account;
    public DepositHandler(Account account) => this.account = account;
    public void Handle(DepositCommand command) => account.Deposit(command.Amount);
}
```

The command requests a deposit; the event reports it after the in-memory balance changes. If a synchronous observer throws, the caller sees an exception even though the balance was already changed. This sample has no persistence, transaction, concurrency protection, deduplication, or durable event delivery; those require explicit design.

Pattern review: problem, use, avoid, example

Pattern	Problem and when to use	When not to use	Illustrative example
Factory	Centralize nontrivial object creation or runtime selection.	Direct construction is simple and stable.	Select a configured shipping-price implementation.
Strategy	Swap algorithms behind one behavioral contract.	A small stable conditional is clearer.	Choose standard or express shipping calculation.
Repository	Express a meaningful data-access boundary around domain queries.	A generic wrapper merely duplicates the ORM.	Find open orders through an order-specific contract.
Unit of Work	Coordinate changes as one persistence operation.	An extra wrapper adds nothing over an existing ORM context.	Persist related order changes using one `DbContext`.

Pattern	Problem and when to use	When not to use	Illustrative example
Decorator	Add behavior around the same contract compositionally.	The wrapper introduces misleading semantics or needless layers.	Add timing around a price reader.
Adapter	Translate a mismatched external interface or representation.	The existing interface already fits the client.	Convert a legacy cents-based price API to a decimal price contract.
Facade	Give a client a focused entry point to a complex subsystem.	The subsystem is already simple or the facade becomes a god object.	Expose a quote operation over pricing collaborators.
Observer	Notify multiple interested in-process subscribers of a change.	Direct control flow or durable delivery is required instead.	Raise an in-process balance-changed event for a UI.
Command	Represent an action explicitly for handling, validation, or scheduling.	A direct call is enough and no request object is useful.	Pass a deposit request to a handler.

These examples demonstrate the patterns, not Sai's actual FinanceHub implementation.

Real-world tie-in: FinanceHub

Reflect on the roadmap's prompt: **Which pattern shows up naturally in FinanceHub? Name it without forcing it.** Start with a workflow or design decision that actually exists in your FinanceHub idea or implementation. Identify the problem, the concrete roles, why the pattern helps, its costs, and an alternative. If there is no natural example yet, say that honestly; a pattern should describe a useful design, not be added merely to claim experience.

Model answer shape: “In FinanceHub, [actual workflow or intended component] needs to [real requirement]. I would use / used [pattern] because [specific problem and roles]. The alternative was [plausible alternative], and the tradeoff is [complexity, consistency, or operational cost]. [What is implemented versus only planned.] I would verify the decision with [relevant test or evidence].”

Likely interview questions

1. How does an event differ from a command, and what do those names not guarantee?
2. What happens if an in-process event handler throws, and how can subscriptions affect object lifetime?
3. Why is a C# event not a durable message queue?
4. What is needed before a command can be retried safely?
5. How do Factory and Strategy differ, and when might they work together?
6. How would you decide whether a repository or unit-of-work wrapper adds value over EF Core?
7. Distinguish Decorator, Adapter, and Facade using one concrete integration.
8. Which pattern genuinely fits a FinanceHub workflow, and what is its downside?

Common pitfalls / misconceptions

- Calling a command undoable or idempotent merely because it is an object.

- Treating synchronous in-process events as reliable cross-process messaging.
- Naming a pattern without explaining the problem, alternatives, or cost.
- Claiming planned FinanceHub components are already implemented or measured.

Cheat-sheet recap

- Observer: notify interested subscribers; manage failures and subscription lifetime.
- Command: represent an action; explicitly design retries, durability, and undo if needed.
- Explain every pattern as **problem** → **fit** → **tradeoff** → **example**, including when not to use it.
- Use FinanceHub facts or clearly labeled plans, never invented implementation details.